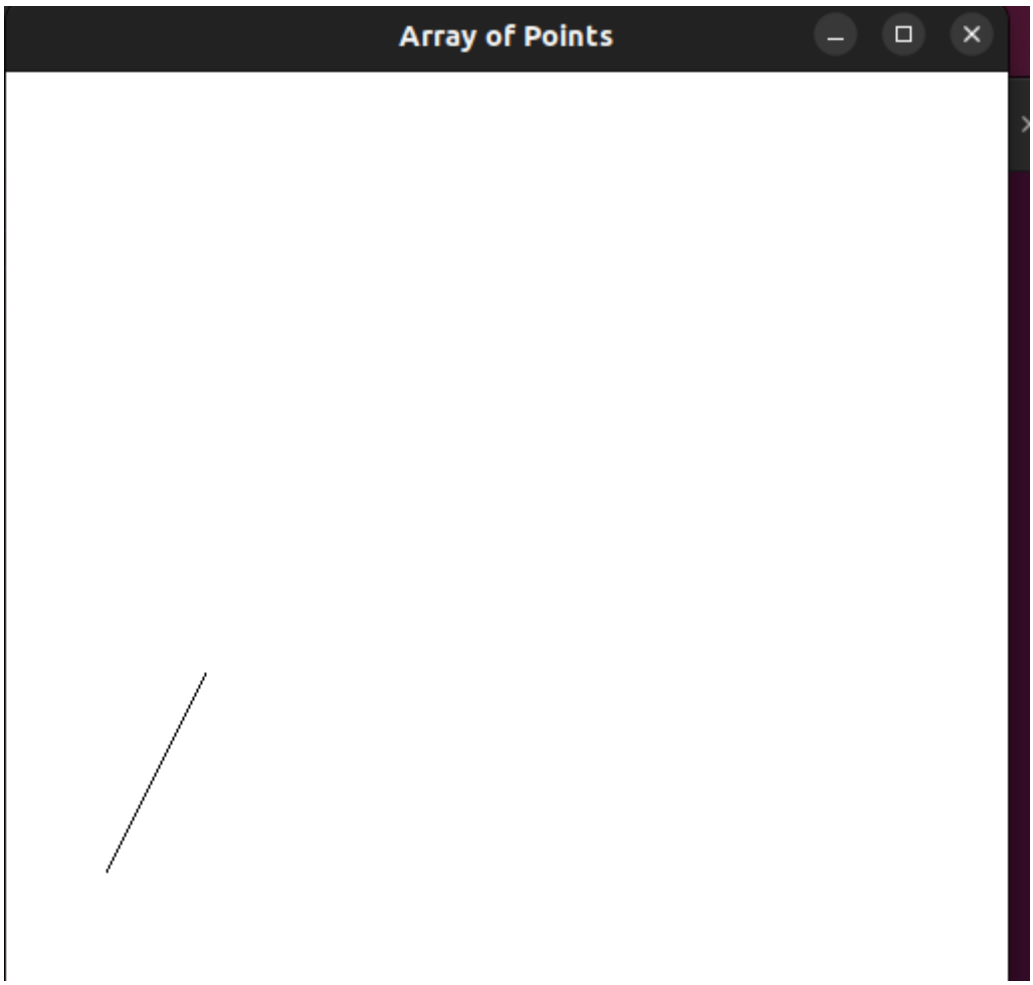
Lab 2

Q1. Input the points as array int point [3][2]={ {50, 100}, {75, 150}, {100, 200}}. Use loops to traverse and draw the lines.

```
#include <GL/glut.h>

int points[3][2] = {{50,100},{75,150},{100,200}};

void init() {
   glClearColor(1,1,1,1);
   glColor3f(0,0,0);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0,500,0,500);
}

void display() {
   glClear(GL_COLOR_BUFFER_BIT);

   glBegin(GL_LINE_STRIP);
   for(int i=0;i<3;i++)
      glVertex2iv(points[i]);
   glEnd();

   glFlush();
}

int main(int argc,char** argv) {
   glutInit(&argc,argv);
   glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
   glutInitWindowSize(500,500);
   glutCreateWindow("Array of Points");
   init();
   glutDisplayFunc(display);
   glutMainLoop();
   return 0;
}
```

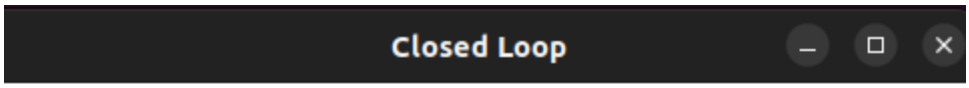
Q2. Define an array of points and connect them in

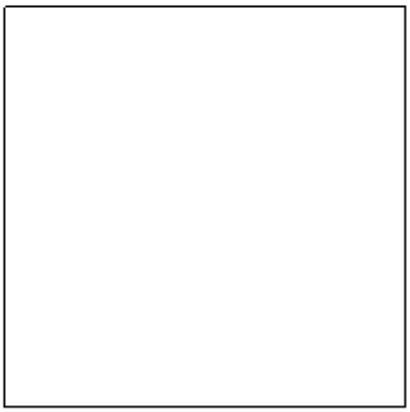


sequence using lines. Ensure that last point connects back to first point to form a closed loop.

```
#include <GL/glut.h>

int points[4][2] = {{100,100},{300,100},{300,300},{100,300}};

void init() {
   glClearColor(1,1,1,1);
   glColor3f(0,0,0);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0,500,0,500);
}

void display() {
   glClear(GL_COLOR_BUFFER_BIT);

   glBegin(GL_LINE_LOOP);
   for(int i=0;i<4;i++)
      glVertex2iv(points[i]);
   glEnd();

   glFlush();
}

int main(int argc,char** argv) {
```

```
    glutInit(&argc,argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(500,500);
    glutCreateWindow("Closed Loop");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

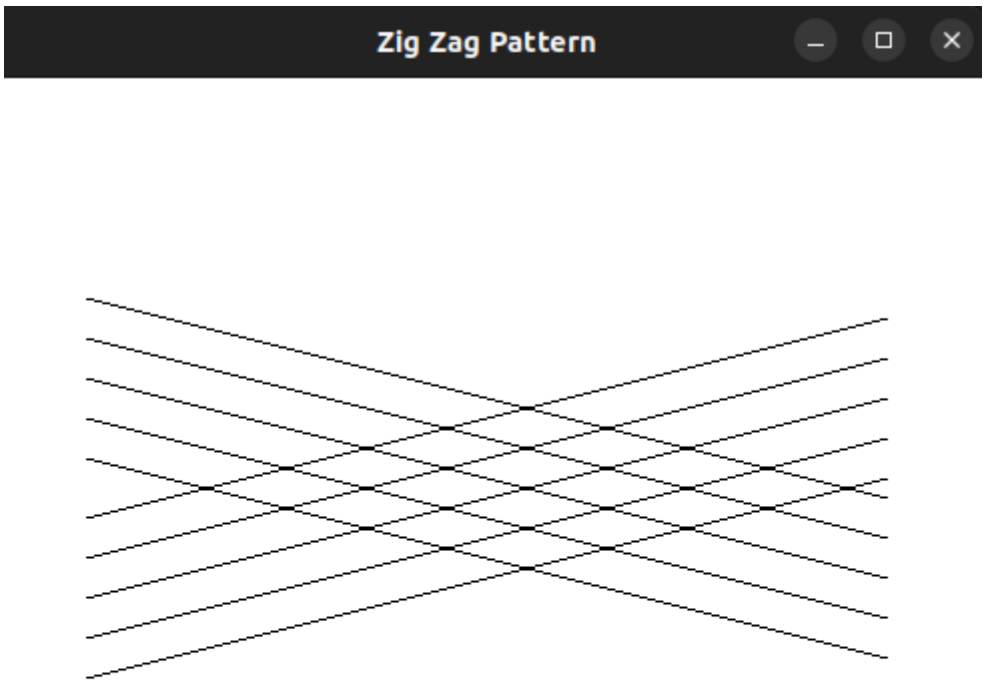




Q3. Use a loop to draw zigzag using lines. Alternate endpoints of the lines between two horizontal levels.

```
#include <GL/glut.h>

void init() {
```

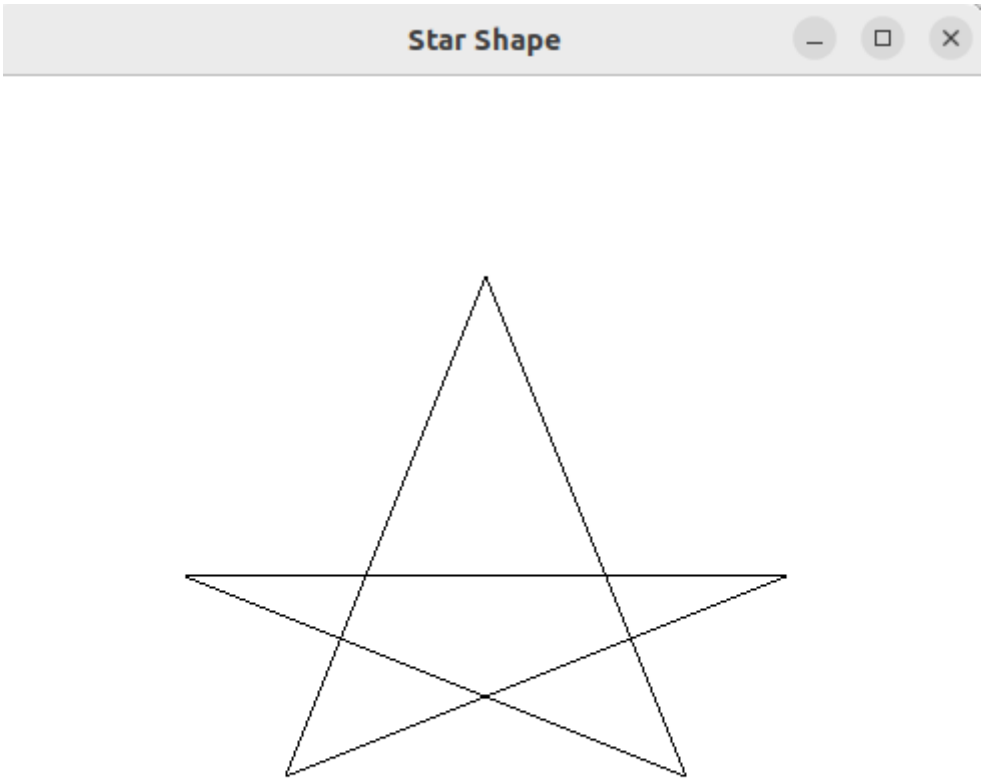
```
    glClearColor(1,1,1,1);
    glColor3f(0,0,0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0,500,0,500);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    int x1 = 50, x2 = 450;
    int y1 = 200, y2 = 300;

    glBegin(GL_LINES);
    for(int i=0;i<10;i++) {
        if(i%2==0) {
            glVertex2i(x1,y1);
            glVertex2i(x2,y2);
        } else {
            glVertex2i(x1,y2);
            glVertex2i(x2,y1);
        }
        y1 += 10;
        y2 += 10;
    }
    glEnd();

    glFlush();
}

int main(int argc,char** argv) {
    glutInit(&argc,argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(500,500);
    glutCreateWindow("Zig Zag Pattern");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```



Q4. Draw a star shape by connecting specific points.

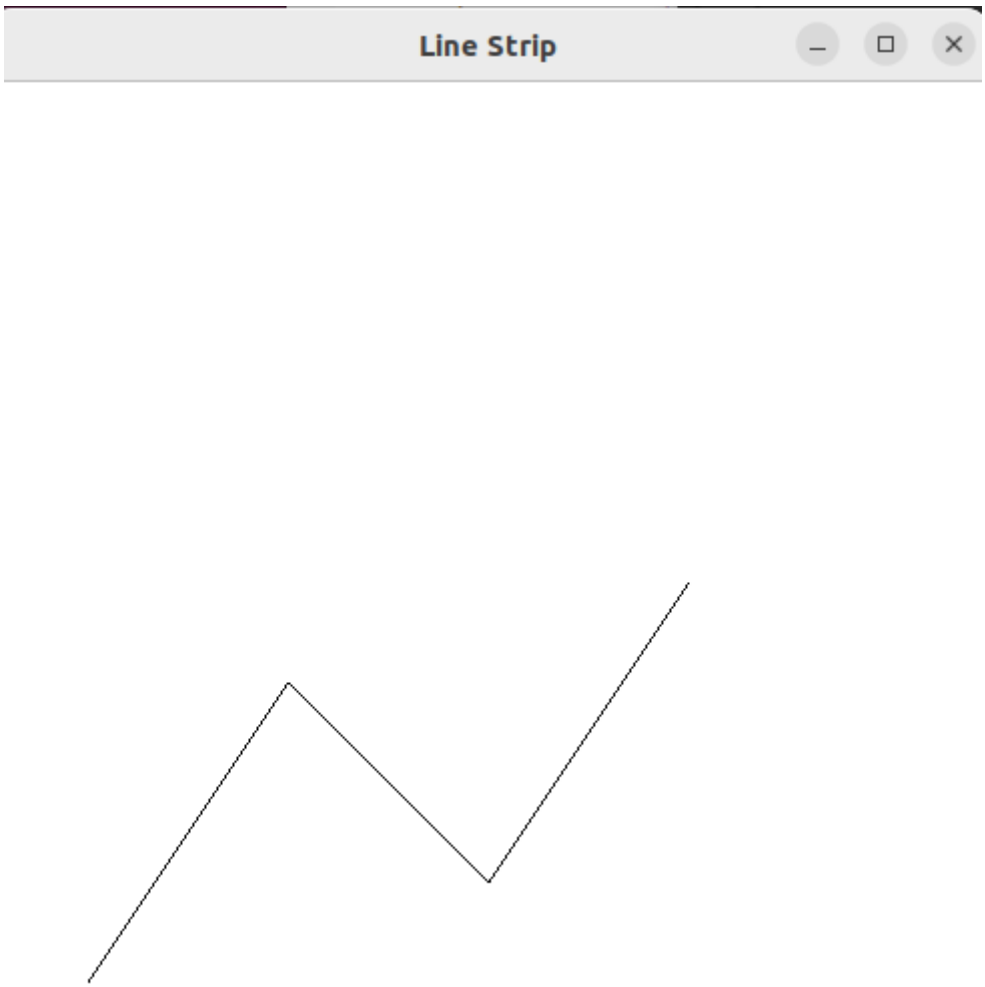
```
#include <GL/glut.h>

void init() {
   glClearColor(1,1,1,1);
   glColor3f(0,0,0);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0,500,0,500);
}

void display() {
   glClear(GL_COLOR_BUFFER_BIT);

   glBegin(GL_LINES);
   glVertex2i(250,400); glVertex2i(150,150);
   glVertex2i(250,400); glVertex2i(350,150);
   glVertex2i(150,150); glVertex2i(400,250);
```

```
    glVertex2i(400,250); glVertex2i(100,250);
    glVertex2i(100,250); glVertex2i(350,150);
    glEnd();

    glFlush();
}

int main(int argc,char** argv) {
    glutInit(&argc,argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(500,500);
    glutCreateWindow("Star Shape");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```



Q5. Implement program to draw line strip by connecting multiple points in sequence. Use an array to store the points and loop through them to render the strip.

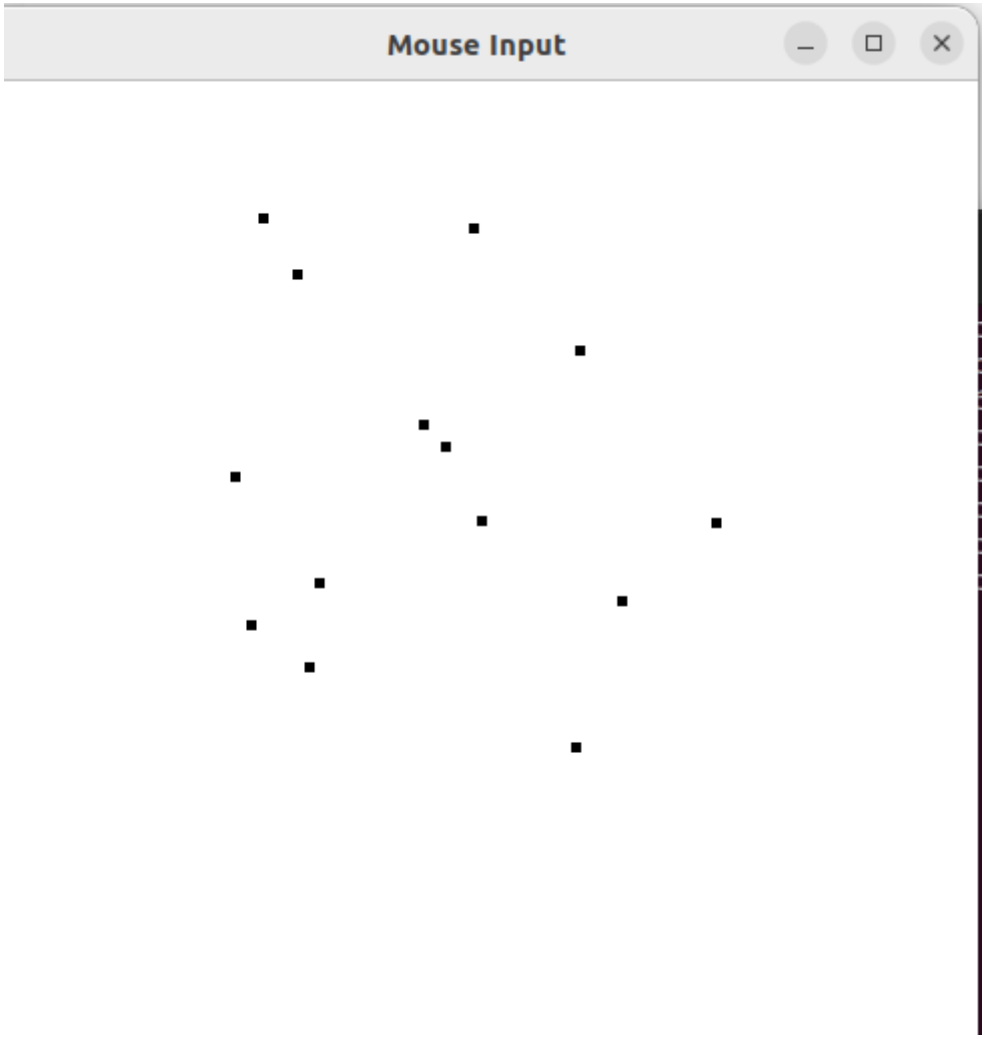
```
#include <GL/glut.h>

int pts[4][2] = {{50,50},{150,200},{250,100},{350,250}};

void init() {
   glClearColor(1,1,1,1);
   glColor3f(0,0,0);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0,500,0,500);
}

void display() {
   glClear(GL_COLOR_BUFFER_BIT);

   glBegin(GL_LINE_STRIP);
   for(int i=0;i<4;i++)
      glVertex2iv(pts[i]);
   glEnd();

   glFlush();
}

int main(int argc,char** argv) {
   glutInit(&argc,argv);
   glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
   glutInitWindowSize(500,500);
   glutCreateWindow("Line Strip");
   init();
   glutDisplayFunc(display);
   glutMainLoop();
   return 0;
}
```



Q6. Allow the user to click on the window to place points dynamically. Use glutMouseFunc() to handle mouse input and plot the points in the clicked locations.

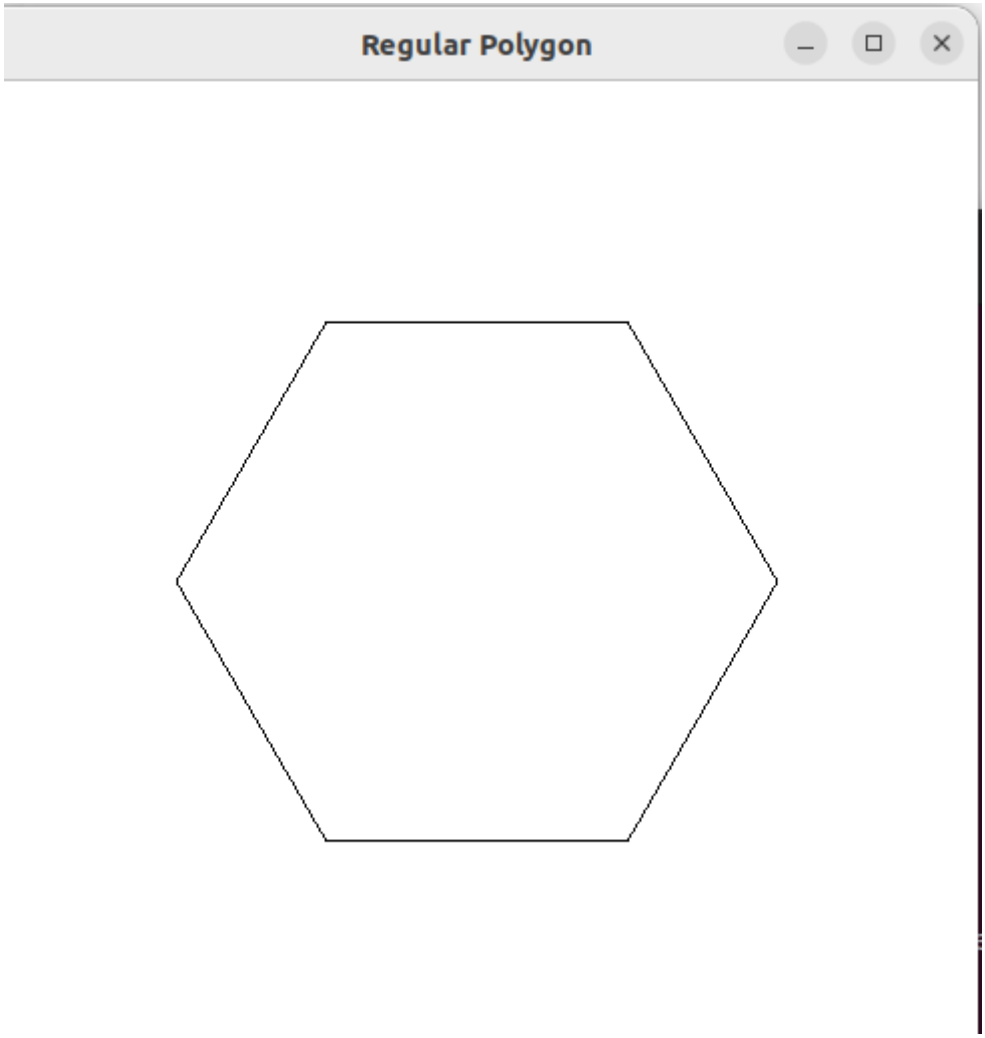
```
#include <GL/glut.h>

void init() {
   glClearColor(1,1,1,1);
   glColor3f(0,0,0);
   glPointSize(5);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0,500,0,500);
}

void mouse(int button,int state,int x,int y) {
   if(button==GLUT_LEFT_BUTTON && state==GLUT_DOWN) {
      glBegin(GL_POINTS);
      glVertex2i(x,500-y);
      glEnd();
      glFlush();
   }
}
```

```
void display() {
   glClear(GL_COLOR_BUFFER_BIT);
   glFlush();
}

int main(int argc,char** argv) {
   glutInit(&argc,argv);
   glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
   glutInitWindowSize(500,500);
   glutCreateWindow("Mouse Input");
   init();
   glutDisplayFunc(display);
   glutMouseFunc(mouse);
   glutMainLoop();
   return 0;
}
```



Q7. Wap to draw regular polygon using loops. Allow the user to specify the no of sides. Hint : Use trigonometry to calculate the vertices of the polygon.

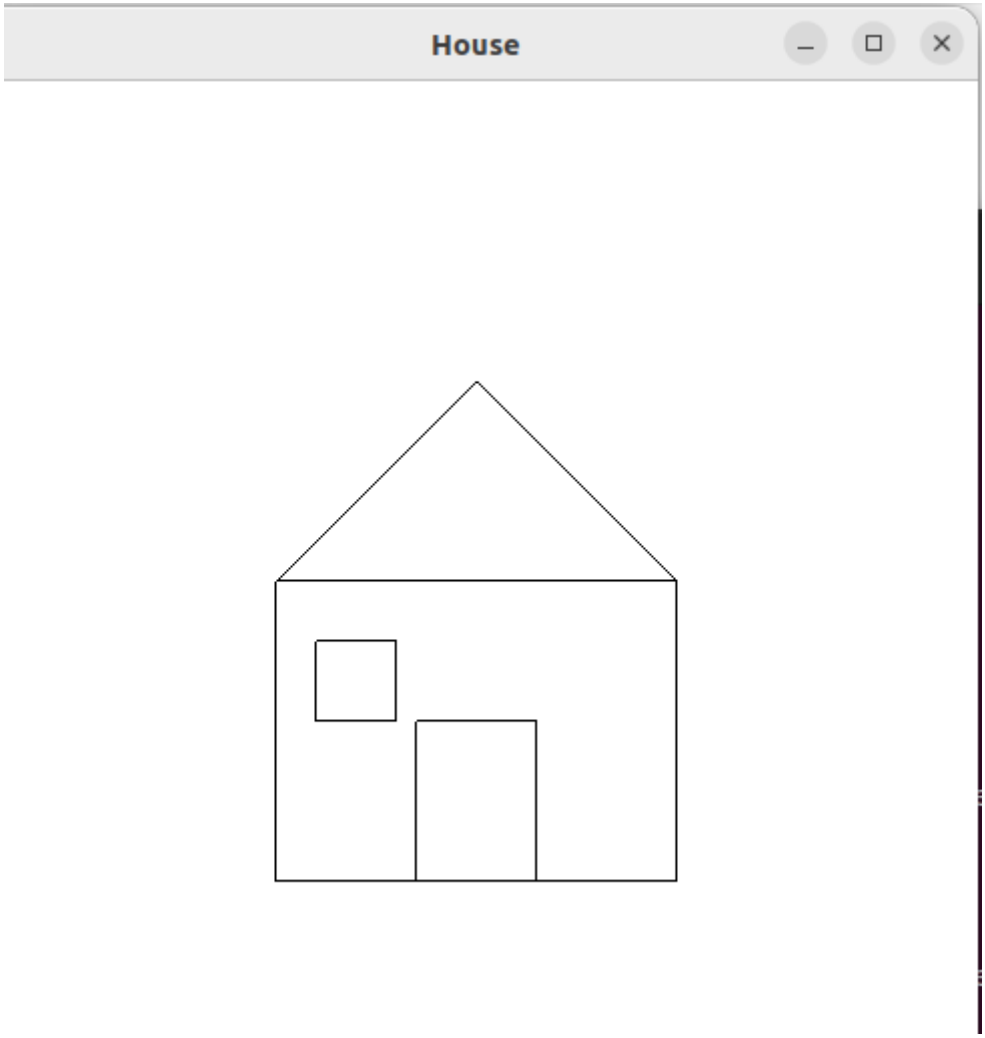
```
#include <GL/glut.h>
#include <math.h>
#include <stdio.h>

int n;

void init() {
   glClearColor(1,1,1,1);
   glColor3f(0,0,0);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0,500,0,500);
}

void display() {
   glClear(GL_COLOR_BUFFER_BIT);

   float r = 150, cx = 250, cy = 250;

   glBegin(GL_LINE_LOOP);
   for(int i=0;i<n;i++) {
      float angle = 2 * M_PI * i / n;
      glVertex2f(cx + r*cos(angle), cy + r*sin(angle));
   }
   glEnd();

   glFlush();
}

int main(int argc,char** argv) {
   printf("Enter number of sides: ");
   scanf("%d",&n);

   glutInit(&argc,argv);
   glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
   glutInitWindowSize(500,500);
   glutCreateWindow("Regular Polygon");
   init();
   glutDisplayFunc(display);
   glutMainLoop();
   return 0;
}
```



Q8. Combine multiple primitives to create a simple scene, such as a house with roof , door and windows using points and lines.

```
#include <GL/glut.h>

void init() {
   glClearColor(1,1,1,1);
   glColor3f(0,0,0);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0,500,0,500);
}

void display() {
   glClear(GL_COLOR_BUFFER_BIT);


   glBegin(GL_LINE_LOOP);
   glVertex2i(150,100);
   glVertex2i(350,100);
   glVertex2i(350,250);
```

```
    glVertex2i(150,250);
    glEnd();


    glBegin(GL_LINES);
    glVertex2i(150,250); glVertex2i(250,350);
    glVertex2i(250,350); glVertex2i(350,250);
    glEnd();


    glBegin(GL_LINE_LOOP);
    glVertex2i(220,100);
    glVertex2i(280,100);
    glVertex2i(280,180);
    glVertex2i(220,180);
    glEnd();


    glBegin(GL_LINE_LOOP);
    glVertex2i(170,180);
    glVertex2i(210,180);
    glVertex2i(210,220);
    glVertex2i(170,220);
    glEnd();

    glFlush();
}

int main(int argc,char** argv) {
    glutInit(&argc,argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(500,500);
    glutCreateWindow("House");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```


House